ElectroAdvisor AI

Complete Technical Architecture & HR / Interview Defense Guide

Candidate: Abhishek
Date: 21 October 2026

40 / 40 TESTS PASSING | FARM STACK | PLAYWRIGHT VERIFIED

🎯 1. Executive Summary & 30-Second Elevator Pitch

The 30-Second Interview Pitch:
"I built **ElectroAdvisor**, an intelligent conversational sales assistant for electronics. When customers shop for tech, they know their practical goals—like '*I need a laptop for college machine learning experiments and campus commute on an ₹80,000 budget*'—but rarely know the specific hardware specifications they need. Generic LLMs often hallucinate prices or fake specs, while ecommerce filters are dumb and rigid. ElectroAdvisor bridges this gap by combining an asynchronous **FastAPI** backend and **React** frontend with a stateful recommendation agent. It translates natural language goals into strict hardware rules, enforces hard budget caps in deterministic Python code so it never overspends, and recommends the **top 2 products** with explicit trade-off comparisons grounded in a real 50-item database."

❌ Why Naive LLMs & Chatbots Fail

- **Hallucinations:** LLMs invent fake model names, discontinued laptops, or fictitious prices.
- **Budget Violations:** Prompts like "around 80k" frequently output ₹95k or ₹1.2L items.
- **Excessive Interrogation:** Chatbots ask 15–20 robotic survey questions in a row.
- **API Fragility:** Vulnerable to network stalls, rate limits, and quota tireout.

✅ How ElectroAdvisor Solves This

- **Strict Separation:** LLM only extracts structured entities; deterministic code filters & scores.
- **Deterministic Budget Caps:** Hard constraint eliminates over-budget items before scoring.
- **Decision Graph:** Evaluates information sufficiency; stops questioning once essentials are known.
- **Dual-Mode Failover:** 5.0s timeout with instant fallback to regex rules in <5ms.

🏛️ 2. High-Level Architecture & Technical Stack

ElectroAdvisor complies with the **FARM Stack** (FastAPI, React, SQLite/SQLAlchemy) with clean layer separation:

Architecture Layer	Technologies	Responsibilities & Technical Justification
Client Layer	React 18, Vite, Lucide Icons, Custom CSS Tokens	Chat bubble stream, live State Inspector drawer, dynamic product cards, side-by-side trade-off matrix, catalog and architecture modals.
API Layer	FastAPI, Pydantic v2, Uvicorn, ASGI Async	12 REST endpoints covering catalog pagination, detail lookups, multi-product comparison, category taxonomies, state override, session reset, feedback, and diagnostics.
AI Agent Core	State Manager, Decision Graph, Dual Extractor	Stateful multi-turn memory tracking, hard constraint vs. soft preference separation, mind-change detection, category isolation, and focused follow-up generation.
Recommendation Engine	Two-Stage Filter & Rank, Explainer	Stage 1 hard constraint filtering (budget, RAM, dedicated GPU) $\rightarrow$ Stage 2 multi-factor utility scoring (0–100 pts) $\rightarrow$ Top 2 diversity selection $\rightarrow$ Spec-accurate trade-offs.
Data Layer	SQLite, SQLAlchemy ORM, Pydantic Schemas	Relational persistence for 50 generalised records across laptops, smartphones, tablets, monitors, and headphones. Parameterized queries prevent SQL injection.

🔄 3. The 5-Step Runtime Request Flow

When a user sends a message (e.g., "*Around ₹80,000. Mostly local experiments, and I carry it around campus.*"), the system processes it in five distinct steps:

1. **Step 1: NLP Entity Extraction (`extractor.py`)** — Translates unstructured customer text into a structured Pydantic schema (`budget_max: 80000.0` , `primary_uses: ["machine_learning"]` , `local_ml: True` , `portability: True`). Uses Gemini 1.5 Flash or OpenAI with a strict **5.0s timeout**, falling back to a deterministic regex engine.

2. **Step 2: State Tracking & Mind-Change Reconciliation (`state.py`)** — The `StateManager` merges the new entities into persistent conversation memory. If the customer updates a previously stated requirement (e.g., lowering budget from 80k to 60k), it overwrites the field while preserving earlier context.
3. **Step 3: Decision Graph Orchestration (`orchestrator.py`)** — Evaluates information completeness. If essential fields are missing (e.g., use case or budget), it halts and asks **one focused follow-up question**. If requirements are sufficient, it immediately transitions to recommendation.
4. **Step 4: Two-Stage Filtering & Ranking (`filter_rank.py`)**:
   - *Stage 1 (Hard Constraint Filter):* Automatically disqualifies products where `price > budget_max` or where local ML is requested but dedicated GPU is missing.
   - *Stage 2 (Soft Preference Ranking):* Computes a 0–100 utility score: Use-Case Fit (40 pts) + Portability (20 pts) + Battery (15 pts) + Customer Rating (15 pts) + Value (10 pts). Selects the Top 2 complementary models.
5. **Step 5: Rationale & Trade-Off Generation (`explainer.py`)** — Formats spec-accurate explanations directly from database fields (zero hallucinations) and generates a structured side-by-side trade-off matrix across Graphics, Portability, Battery, and Price.

📁 4. Codebase Deep Dive: What Every File Does

File Path	Core Component	Plain-English Role & Implementation Details
`backend/app/agent/extractor.py`	Dual-Mode Extractor	Extracts structured requirements from natural conversation. Supports Google Gemini and OpenAI with strict 5.0s timeout. Includes regex rule engine for Indian numbering (`₹1,20,000` , `80k` , `1.5 lakh`).
`backend/app/agent/state.py`	State Manager	Maintains multi-turn conversational memory. Merges incremental updates, updates state when user changes budget mid-chat, and resets hardware constraints when switching product categories.
`backend/app/agent/orchestrator.py`	Decision Orchestrator	Evaluates requirement completeness. Prevents 20-question survey loops: asks targeted follow-ups only when category, use case, or budget are unknown, then triggers recommendation immediately.
`backend/app/agent/filter_rank.py`	Filter & Ranking Engine	Stage 1 hard filtering (budget cap, minimum RAM, dedicated GPU) followed by Stage 2 soft preference utility scoring (0–100 points) and Top 2 diversity selection.
`backend/app/agent/explainer.py`	Grounded Explainer	Produces grounded explanations citing real specs and builds trade-off matrices. Generates dynamic no-match explanations citing real catalog prices when constraints cannot be met.
`backend/app/agent/tools.py`	Database Tools	Parameterized SQLAlchemy queries for category, price range, and specification queries. Prevents arbitrary SQL execution and injection attacks.
`backend/app/data/seed_data.py`	Product Catalog	50 generalised records across 5 categories (20 laptops, 8 smartphones, 8 tablets, 7 monitors, 7 headphones) with realistic specs, pros, cons, and Unsplash images.
`backend/app/config.py`	Settings & Config	Centralized Pydantic configuration loading environment variables, CORS settings, and `LLM_TIMEOUT_SECONDS=5.0` .
`frontend/src/App.jsx`	Frontend Controller	Manages message state, API integration, quick prompts, and layout coordination between chat and inspector.
`frontend/src/components/StateInspector.jsx`	Live Memory Inspector	Live drawer showing the active requirements extracted by the agent (category, budget ceiling, required RAM, detected constraints, and current intent).
`frontend/src/components/ProductCard.jsx`	Product Cards	Renders recommendation cards with suitability match score bar, formatted Indian Rupee prices, pros, cons, and key advantages.
`frontend/src/components/TradeOffMatrix.jsx`	Trade-Off Matrix	Side-by-side comparison table highlighting trade-offs between Top 1 and Top 2 recommendations across Compute, Portability, Battery, and Price.
`frontend/verify_playwright.cjs`	E2E Browser Test	Automated Playwright verification script testing headless Chromium launch, sandbox network egress, and local asset rendering (`npm run test:e2e`).

🛡️ 5. Key Edge Cases & Reliability Hardening

Highlights to Mention in Your Interview:

- **50 Generalised Records:** Expanded catalog from 23 to 50 realistic records across 5 categories, verified by automated tests.

- **Strict LLM Timeout Safety:** Added configurable `LLM_TIMEOUT_SECONDS=5.0`. Rather than worrying about API key rotation or exhaustion heuristics, the agent cancels stalled LLM calls after 5s and falls back cleanly to deterministic rules.
- **Playwright Headless Browser Fix:** Diagnosed CDN mirror 404s, installed official Playwright driver binaries (`npx playwright install chromium`), and verified sandbox network egress and browser compatibility via `npm run test:e2e`.
- **Indian Currency Parsing:** Handles `₹1,20,000`, `1,20,000 INR`, `80k`, and `1.5 lakh` without regex crashes.
- **Mind Changes:** Customer can reduce their budget mid-chat (e.g. ₹80k $\rightarrow$ ₹60k); state updates immediately and computes dynamic alternatives.
- **Category Isolation:** Switching from laptop to monitor clears RAM and GPU hard constraints so monitor recommendations aren't blocked.
- **Honest No-Match Fallback:** If a customer requests impossible constraints (e.g. RTX 4090 under 30k), it outputs an honest database-computed explanation with real alternatives instead of hallucinating.

⚠️ 6. Current Limitations & 🚀 7. Future Production Roadmap

⚠️ Current Limitations (Self-Awareness)

- **In-Memory / SQLite State:** State stored in local SQLite / memory. Not yet horizontally scalable across multiple servers.
- **Static Synthetic Catalog:** 50 representative items; not yet integrated with live e-commerce APIs (Amazon/Flipkart).
- **Rule-Based Boundaries:** Offline regex mode handles common queries but lacks deep linguistic nuance for convoluted idioms.
- **Text-Only Modality:** Lacks voice input or spec-sheet image upload.

🚀 Future Roadmap (Production Vision)

- **Distributed Persistence:** Migrate from SQLite to **PostgreSQL**, and use **Redis** for sub-millisecond conversation state caching.
- **Hybrid RAG & Vector Search:** Embed customer reviews and technical manuals using Vertex AI embeddings + pgvector for semantic retrieval.
- **Live Inventory Webhooks:** Stream real-time pricing, flash discounts, and stock levels from ecommerce platforms.
- **Multimodal & Voice:** Add Whisper STT / ElevenLabs TTS for in-store physical kiosk tablets, and image recognition for syllabus spec sheets.

🎯 8. HR & Technical Interview Q&A Cheatsheet

Q1: "Why did you use FastAPI instead of Flask or Django?"

Answer: "FastAPI is natively asynchronous (ASGI), which is critical for AI agents making non-blocking calls to external LLMs and databases. It also provides automatic Pydantic request/response validation and auto-generates interactive Swagger API documentation at `/docs`, saving significant engineering overhead."

Q2: "Why didn't you let the LLM generate SQL queries directly?"

Answer: "Allowing an LLM to generate raw SQL poses severe security risks like SQL injection, and reliability risks like hallucinated tables or malformed syntax. In ElectroAdvisor, tool execution is strictly controlled through parameterized SQLAlchemy ORM queries in `app/agent/tools.py`. The LLM only extracts structured parameters, which are validated before querying."

Q3: "How do you guarantee the AI never recommends products above the customer's budget?"

Answer: "We use a two-stage recommendation pipeline. Maximum budget is enforced as a **hard constraint** in Stage 1. Any product where `price > budget_max` is strictly disqualified in Python code before scoring even begins. An over-budget product can never reach the soft ranking stage regardless of its features."

Q4: "What happens if the Gemini or OpenAI API times out or fails?"

Answer: "As emphasized during our architecture review, we prioritize strict timeout guarantees over API key tireout heuristics. We enforce an async timeout of `LLM_TIMEOUT_SECONDS=5.0`. If an LLM call exceeds 5s, the coroutine is cancelled immediately, logged, and seamlessly falls back to our deterministic rule extractor in less than 5ms with zero disruption to the user."

Q5: "How did you test your system?"

Answer: "We built a test suite of **40 automated pytest tests** covering all 12 endpoints, state memory, mind-changes, currency parsing, and timeout fallbacks. We also wrote an automated **Playwright E2E headless browser test** (`npm run test:e2e`) verifying external network access and local React asset rendering."

📋 9. Step-by-Step Live Demo Script for the Assessment Benchmark

Open `http://localhost:5173` in your browser and run through this exact 3-turn benchmark:

Turn	Customer Natural Input	Agent State & Decision	Expected Agent Output
Turn 1	*"I need a laptop for college."*	Category set to `laptop`. Primary use is missing. Decision: `ask_clarification`.	Asks a focused question: *"What will you mainly use it for: general coursework, programming, gaming, creative work, or machine learning?"* (Does not ask for budget yet).
Turn 2	*"Programming and machine learning. My budget is limited."*	Primary uses: `programming`, `machine_learning`. Budget noted as limited. Decision: `ask_clarification`.	Asks: *"What is your approximate budget? And do you need to run models locally, or will you mostly use cloud platforms?"*
Turn 3	*"Around ₹80,000. Mostly local experiments, and I carry it around campus."*	Budget: `₹80,000`. Hardware: `dedicated GPU`, `RAM >= 16GB`, `portability: True`. Decision: `recommend`.	Delivers Top 2 laptops strictly under ₹80k: **Acer Nitro V 15** (₹76,990) & **ASUS TUF A15** (₹79,990) with RTX 4050 6GB, plus side-by-side Trade-Off Matrix. Stops questioning!
Bonus Turn	*"Actually, my budget changed to ₹60,000."*	Budget updated to `₹60,000`. Dedicated GPU requirement persists. Decision: `no_match`.	Honest explanation: Dedicated GPU laptops start at ₹62,990 (HP Victus). Suggests cloud ML or budget adjustment.

✅ Pre-Interview Verification & Quick Commands

- **Backend API:** `http://127.0.0.1:8000` (Swagger docs at `/docs`, health at `/api/v1/health`)
- **React Frontend:** `http://127.0.0.1:5173` (Interactive State Inspector, Catalog & Architecture modals)
- **Run Test Suite:** `cd backend && .\.venv\Scripts\python.exe -m pytest` (40 / 40 passing in ~2.1s)
- **Playwright E2E:** `cd frontend && npm run test:e2e` (Headless browser verified with 100% pass)